Working example of TLB, address translation and superpage

Sparsh Mittal

TLB = Translation Lookaside Buffer

Superpage

- A VM page with size = 2^k times system page size and which maps to contiguous physical pages
- Increases coverage by using one entry for multiple virtual pages

Case-1: Page size is 1B (Page offset has 0 bits)

- If I access frame 7, which address I get? Answer: 7
- If I access frame 4, which address I get? Answer: 4

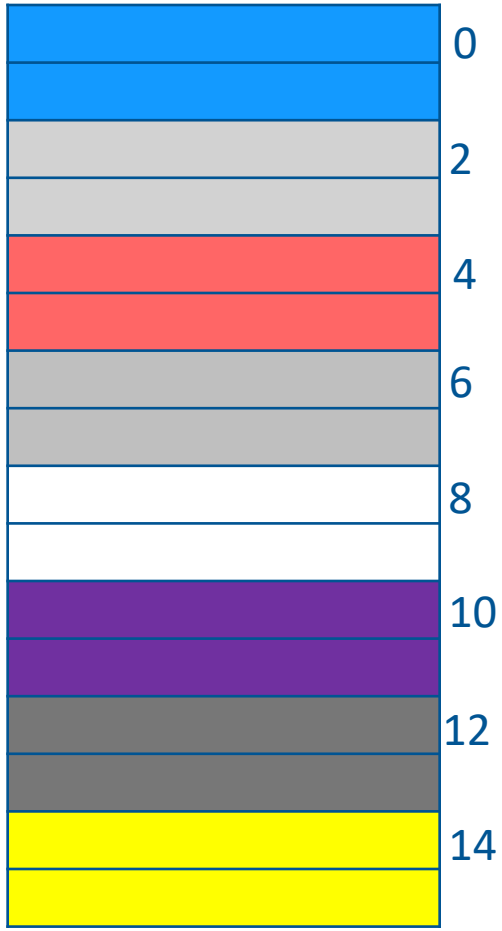
Case-1: Page size is 2B (page-offset has 1 bit)

- If I access frame number 3 which addresses I get (that are part of this frame)? Answer: 6 and 7

Reason: 6=**011**0, 7=**011**1

On ignoring page offset bits, remaining bits (11) are same. Both 6 and 7 are part of the same page.

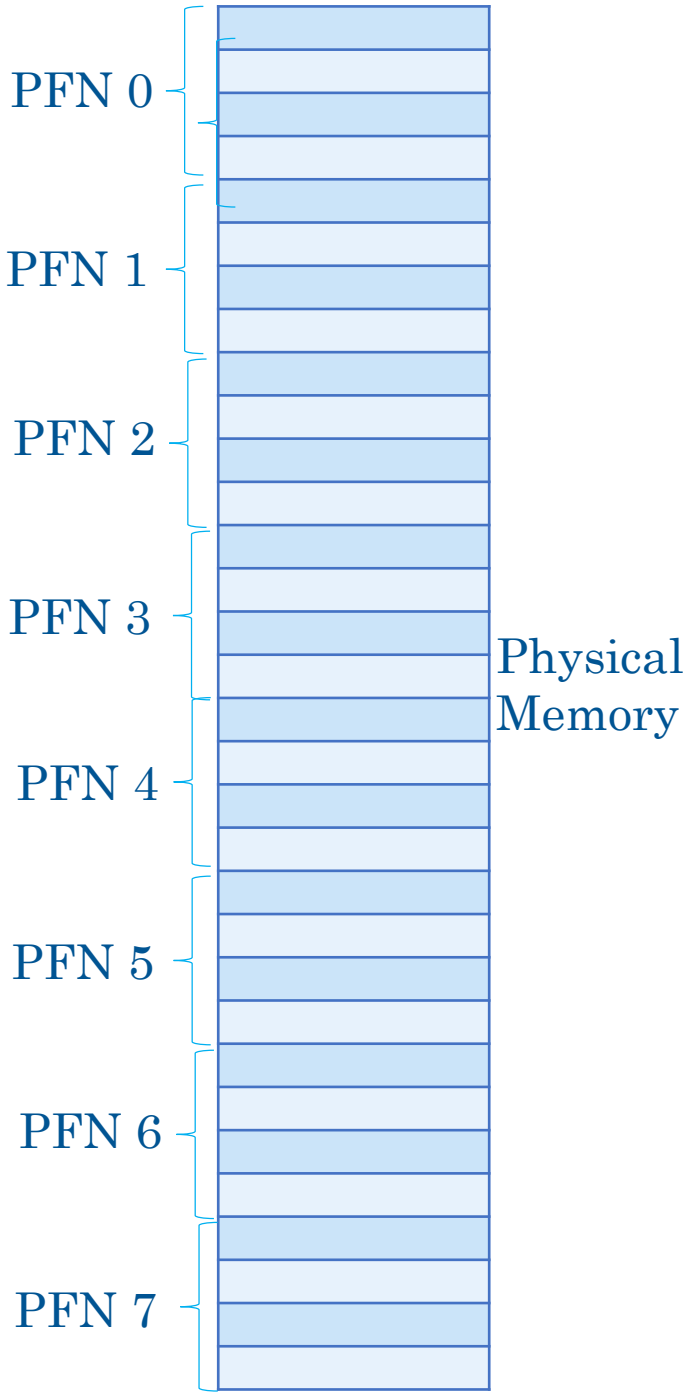
On ignoring page offset bits, remaining bits (11) are nothing but frame number, which is 3.



Example

- Consider a case where our system page-size is 4B (for simplicity). Our total physical memory is only 32B and thus, a physical address is 5b.

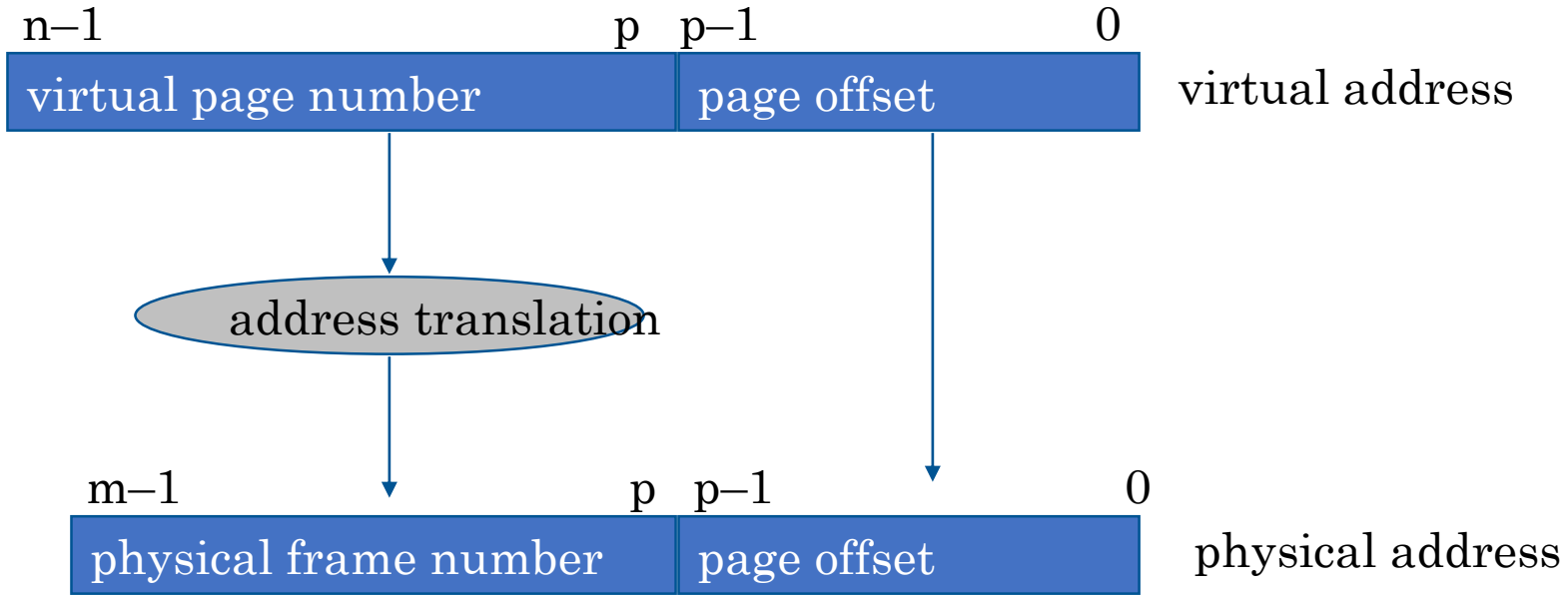
- Our virtual memory is 64B, which means a virtual address is 6b.
- (a) How many physical frames can reside in the physical memory at any point of time?
- Answer: 8
- (b) How many total virtual pages are there?
- Answer: 16



VM Address Translation

- Parameters
  - $P = 2^p$ = page size (bytes).
  - $N = 2^n$ = Virtual-address limit
  - $M = 2^m$ = Physical-address limit

In our example:
m =5, n=6, p=2



Page offset bits don't change as a result of translation

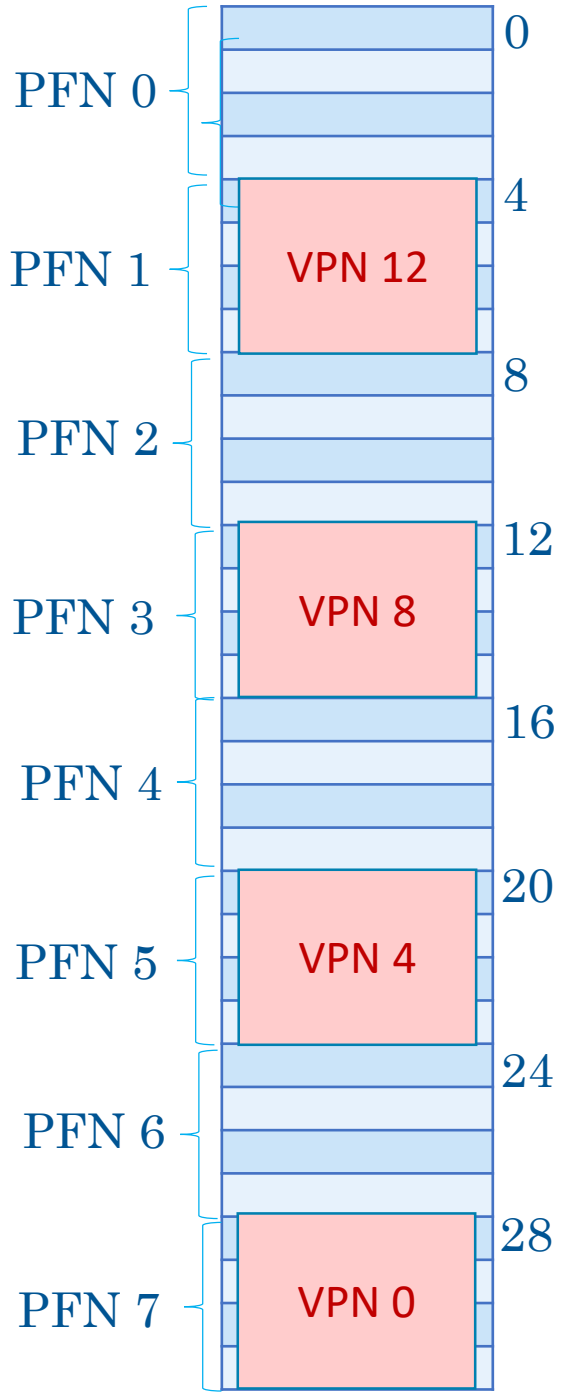
- (c) Which of the two statements are correct? Page table stores mapping between
- (i) virtual address to physical address
- (ii) virtual page number to physical frame number
- Answer: (ii) is correct.

- (d) Assume that at a point of time, the page table has following mappings:

Page Table

VPN	PFN
0	7
4	5
8	3
12	1

- Clearly, PFNs 0, 2, 4, 6 have not yet been assigned to any virtual page. Find their corresponding byte-address ranges in memory?
- Answer: 0-3B, 8-11B, 16-19B, 24-27B



- (d) Assume that at a point of time, the page table has following mappings:

Page Table

VPN	PFN
0	7
4	5
8	3
12	1

- Question: What is physical address corresponding to VA 100011_2.
- Answer: 01111. Steps to obtain answer:

1. First remove 2 page-offset bits.
2. This gives us VPN, which is 1000.
3. Access page table, which gives us PFN of 011
4. Concatenate page-offset bits to get 011**11**

- (e) The TLB has 2 entries. Lets call it TLB1. The processor accesses the following virtual pages in sequence: 12, 8, 4, 0

- At the end of this sequence, which VPN-PFN entries will be stored in the TLB1 if the TLB1 uses LRU replacement policy.
- Answer: 4, 0. Here 0 is MRU.

- (f) What is the reach of this TLB1?
- Answer: 8B. (=2 entries * pageSize of 4B)

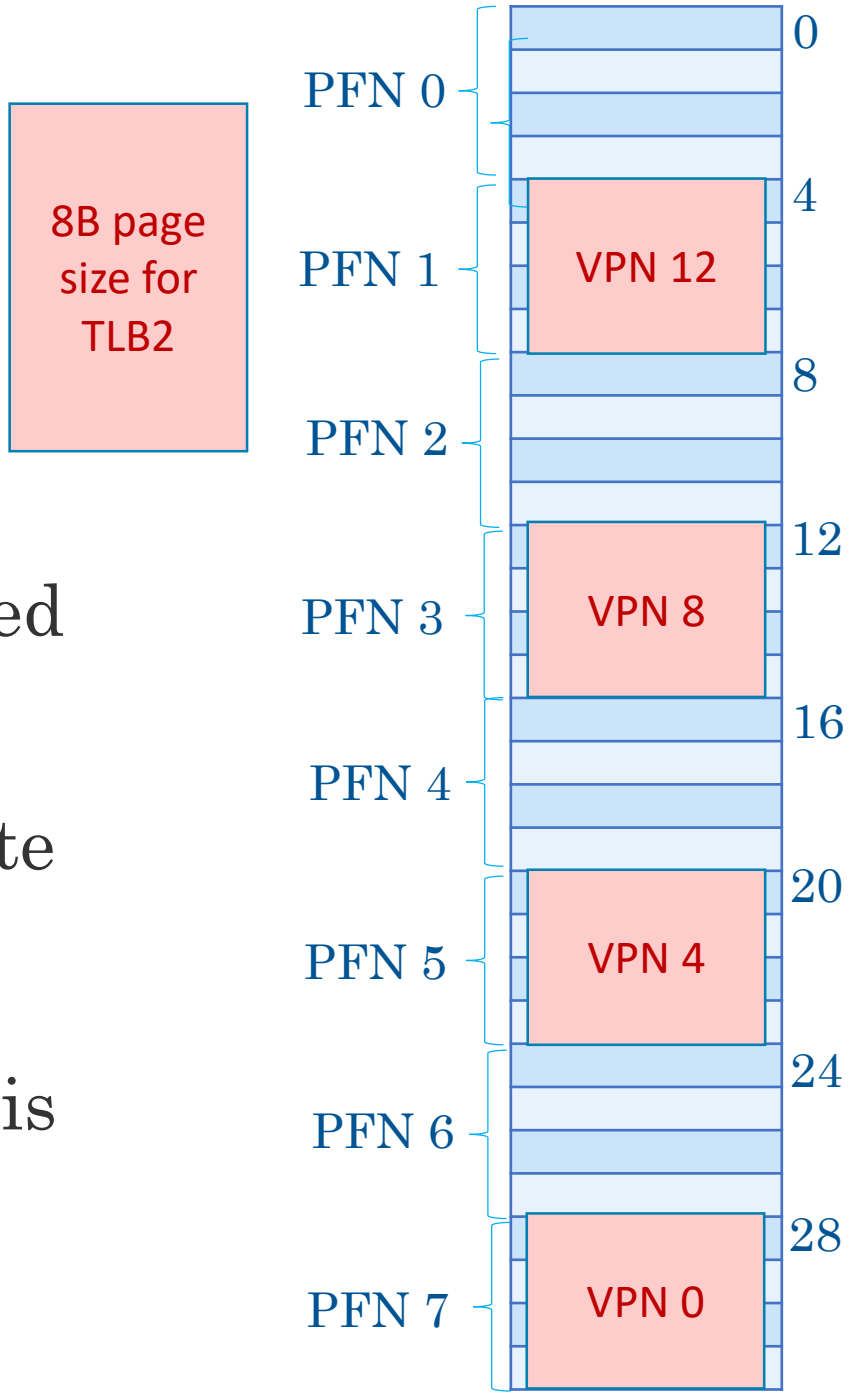
Page Table

VPN	PFN
0	7
4	5
8	3
12	1

TLB1

VPN	PFN
0	7
4	5

- (g) We add another TLB, called TLB2 to this system, which works for a page-size of 8B. Clearly, it uses a super-page. It also has 2 entries. What would be its reach?
- Answer: 16B.
- (h) See the answer to (d). From the unassigned physical address space, can you allocate any physical frame to a virtual page in TLB2 (which uses super-page size of 8B)? If so, write the physical address range of the physical frame that you have assigned.
- Answer: No, we cannot assign because there is no contiguous 8B space available.



- (i) Instead of TLB2, can you allocate a new frame to a virtual page in TLB1?

- Answer: Yes, because a contiguous space of 4B is available.

- Take home message: Use of large page size increases the TLB reach but also makes it difficult to find contiguous region of size equal to large-page size.

Deciding best page size

- (j) Application1 accesses following Virtual addresses: 0, 1, ..31
- Application2 accesses following Virtual addresses: 0, 1, 2, 3, 12, 13, 14, 15
- Which page size is best for them. Note: our options are 4B or 8B, because our TLBs can only support one of the two page sizes.
- Reasoning: Goal: wastage of memory should be minimum and number of TLB entries should be minimum.
- App1: With 8B, #Entries=4 and wastage=0.
- With 4B, #Entries=8 and wastage =0
- ➔ For App1: 8B is better.
- App2: With 4B, #Entries=2, wastage=0.
- With 8B, #Entries=2, wastage=8B
- ➔ For app2, 4B is better.

TLB in real processors

Out of course

TLB architecture in Older Processors

Processor	Entries	Assoc.	Year
21064	32I + 32D	fully	1992
Super Sparc	64U	fully	1992
R4000	96U	fully	1992
PA-7100	120U	fully	1992
PowerPC 601	256U	2 way	1993
R8000 (TFP)	384U	3 way	1994
PowerPC 620	64I + 64D	fully	1995
21164	48I + 64D	fully	1995
R10000	8I + 64D	fully	1996
Strong ARM	32I+ 32D	fully	1996

I= instruction, D = data, U = unified

Juan et al. ISLPED'97

Out of course

D-TLB Architecture in Recent Processors

Processor	L1 TLB Configuration	L2 TLB Configuration
Intel Haswell [5]	4-way SA split L1 TLBs: 64-entry (4KB), 32-entry (2MB) and 4-entry (1GB)	8-way SA 1024-entry (4KB and 2MB)
AMD 12h family [6]	48-entry FA TLB (all page sizes)	4-way SA 1024-entry TLB (4KB) 2-way SA 128-entry TLB (2MB) 8-way SA 16-entry TLB (1GB)
Sparc T4 [3]	128-entry FA TLB (all page sizes)	
UltraSparc III	2-way SA 512-entry TLB (8KB) 16-entry FA TLB (superpages and locked 8KB)	

SA = set-associative

Out of course

Papadopoulou et al. HPCA'15

Private, per-core, data TLB hierarchy in 3 recent Intel processors

L1 DTLBs

Sandy Bridge / Haswell / Broadwell		
Page-size	Entries	Assoc.
4 KB	64	4-way
2 MB	32	4-way
1 GB	4	fully

L2 DTLBs

Sandy Bridge			Haswell			Broadwell		
Page-size	Entries	Assoc.	Page-size	Entries	Assoc.	Page-size	Entries	Assoc.
4 KB	512	4-way	4 KB/2 MB	1024	8-way	4 KB/2 MB	1536	n/a
2 MB	—							
1 GB	—		1 GB	—		1 GB	16	n/a

Karakostas et al. HPCA'16